



AI 서비스를 위한 SW 기초 Full-stack Engineering

스마트 데이터 이해 및 활용

day02_종합실습

2026.07

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

- 
1. 학사관리 시스템 - DDL,DML
 - 파일제공
 2. 학사관리 시스템 DQL 8문제 해설
 - JOIN · NULL · CASE · 날짜 · 집계 · CTE
 3. 학사관리 시스템 DQL 5문제 해설
 - VIEW · MATERIALIZED VIEW · ROLLUP · CUBE
 4. 통합 인사정보 관리 시스템(HRIS) - DDL,DML
 - 파일제공
 5. 통합 인사정보 관리 시스템(HRIS) - 종합문제
 6. 제출방법
 - Slack – 다이렉트메세지
 - day02_캡쳐_작성자명.pdf
 - day02_쿼리_작성자명.sql

SQL은 “문장”보다 “데이터의 흐름”으로 이해합니다

학사관리 시스템 DQL 8문제 해설

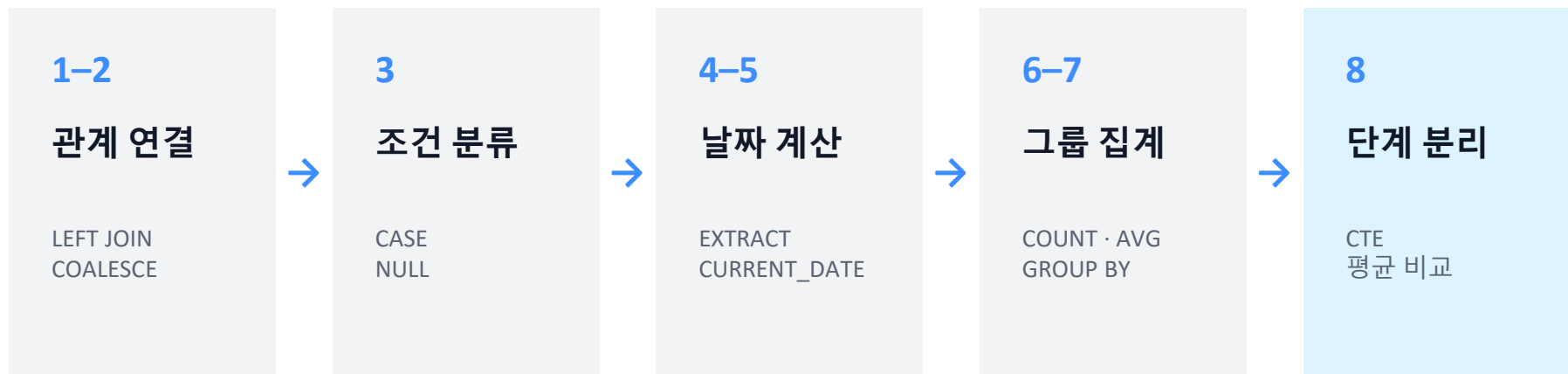
JOIN · NULL · CASE · 날짜 · 집계 · CTE

8

PRACTICE
QUERIES

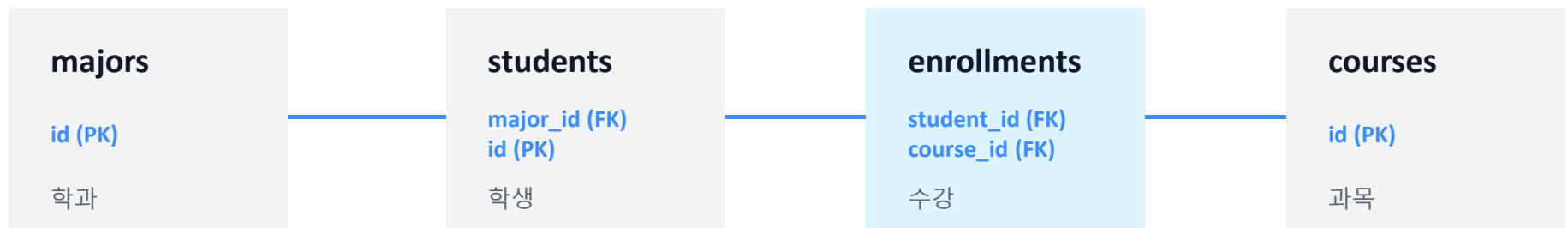
PostgreSQL 13+
app schema

8문제는 하나의 분석 흐름으로 연결됩니다



학습 목표: 결과를 외우는 것이 아니라 “왜 이 JOIN과 함수가 필요한지” 설명할 수 있어야 합니다.

먼저 네개의 테이블의 연결 열을 찾습니다



JOIN을 쓰기 전에 “기준 테이블”과 “연결 키”를 먼저 말해보세요.

- 학생 → 학과: `students.major_id = majors.id`
- 수강 → 학생: `enrollments.student_id = students.id`
- 수강 → 과목: `enrollments.course_id = courses.id`

수업은 문제와 솔루션을 한 쌍으로 진행합니다



각 솔루션 슬라이드에는 실행 가능한 전체 쿼리가 포함됩니다.

문제 1 — 학생별 소속 학과 조회

모든 학생의 학과 정보를 조회하고 미배정 학생도 포함합니다.

조회 결과

- 학번 · 학생명 · 학년
- 학과코드 · 학과명

작성 조건

- students가 기준
- LEFT JOIN 사용
- 학번 오름차순

제공파일 :

day02_postgresql_app_students_enrollments_샘플_스키마.sql

day02_postgresql_app_students_enrollments_DQL_문제01_8문항_학습용.sql

문제 1 솔루션 — 학생별 소속 학과 조회

```
SELECT
  s.student_no AS 학번,
  s.name AS 학생명,
  s.grade AS 학년,
  m.code AS 학과코드,
  m.name AS 학과명
FROM students s
LEFT JOIN majors m
  ON m.id = s.major_id
ORDER BY s.student_no;
```

설명 포인트

- LEFT JOIN은 왼쪽 학생을 모두 보존
- 미배정 학생의 학과 열은 NULL
- INNER JOIN이면 두 학생이 누락

문제 2 — COALESCE로 학과 미배정 처리

NULL인 학과명을 수강생이 읽기 쉬운 문구로 바꿉니다.

조회 결과

- 학번 · 학생명
- 학과명

작성 조건

- LEFT JOIN 사용
- COALESCE 사용
- 학번 오름차순

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 2 솔루션 — COALESCE로 학과 미배정 처리

```
SELECT
  s.student_no AS 학번,
  s.name AS 학생명,
  COALESCE(m.name, '학과 미배정') AS 학과명
FROM students s
LEFT JOIN majors m
  ON m.id = s.major_id
ORDER BY s.student_no;
```

설명 포인트

- m.name이 있으면 원래 학과명
- m.name이 NULL이면 대체 문구
- 행을 제거하지 않고 표시만 변경

문제 3 — CASE로 학점 등급 계산

각 수강 점수를 A~F 및 미입력으로 분류합니다.

조회 결과

- 학생명 · 과목명
- 점수 · 등급

작성 조건

- 3개 테이블 JOIN
- NULL을 먼저 검사
- 높은 점수부터 CASE 작성

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 3 솔루션 — CASE로 학점 등급 계산

```
SELECT
  s.name AS 학생명, c.name AS 과목명, e.score AS 점수,
  CASE
    WHEN e.score IS NULL THEN '미입력'
    WHEN e.score >= 90 THEN 'A'
    WHEN e.score >= 80 THEN 'B'
    WHEN e.score >= 70 THEN 'C'
    WHEN e.score >= 60 THEN 'D'
    ELSE 'F'
  END AS 등급
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id
ORDER BY s.name, c.name;
```

설명 포인트

- CASE는 첫 참 조건에서 종료
- 95점은 90점 조건에서 A
- 낮은 기준부터 쓰면 오판정

문제 4 — 수강 연도·월과 학기 구분

수강신청일에서 연도와 월을 추출해 학기를 판정합니다.

조회 결과

- 학생명 · 과목명 · 신청일
- 수강연도 · 수강월 · 학기

작성 조건

- EXTRACT 사용
- 1~6월은 1학기
- 7~12월은 2학기

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 4 솔루션 — 수강 연도·월과 학기 구분

```
SELECT
  s.name AS 학생명, c.name AS 과목명,
  e.enrolled_at AS 수강신청일,
  EXTRACT(YEAR FROM e.enrolled_at)::integer AS 수강연도,
  EXTRACT(MONTH FROM e.enrolled_at)::integer AS 수강월,
  CASE
    WHEN EXTRACT(MONTH FROM e.enrolled_at)
      BETWEEN 1 AND 6
      THEN '1학기'
    ELSE '2학기'
  END AS 학기구분
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id
ORDER BY s.name, e.enrolled_at, c.name;
```

설명 포인트

- EXTRACT 결과를 integer로 변환
- 3월은 1학기
- 9월은 2학기

문제 5 — 수강신청 후 경과 일수

수강신청일부터 실행 당일까지 며칠이 지났는지 계산합니다.

조회 결과

- 학생명 · 과목명
- 수강신청일 · 오늘날짜
- 경과일수

작성 조건

- CURRENT_DATE 사용
- date - date 계산
- 경과일수 내림차순

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 5 솔루션 — 수강신청 후 경과 일수

```
SELECT
  s.name AS 학생명,
  c.name AS 과목명,
  e.enrolled_at AS 수강신청일,
  CURRENT_DATE AS 오늘날짜,
  CURRENT_DATE - e.enrolled_at AS 경과일수
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id
ORDER BY 경과일수 DESC, s.name, c.name;
```

설명 포인트

- date - date는 정수 일수
- CURRENT_DATE는 실행일 기준
- 동렬은 학생명·과목명 정렬

문제 6 — 학생별 과목 수와 평균점수

수강 행을 학생 한 명당 한 행으로 집계합니다.

조회 결과

- 학번 · 학생명
- 수강과목수 · 평균점수

작성 조건

- LEFT JOIN으로 전체 학생 포함
- COUNT · AVG · ROUND
- NULLS LAST 정렬

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 6 솔루션 — 학생별 과목 수와 평균점수

```
SELECT
  s.student_no AS 학번,
  s.name AS 학생명,
  COUNT(e.course_id) AS 수강과목수,
  ROUND(AVG(e.score), 2) AS 평균점수
FROM students s
LEFT JOIN enrollments e
  ON e.student_id = s.id
GROUP BY
  s.id, s.student_no, s.name
ORDER BY 평균점수 DESC NULLS LAST, s.student_no;
```

설명 포인트

- COUNT(*)는 0명을 1명으로 셀 수 있음
- COUNT(course_id)는 NULL 제외
- GROUP BY 후 한 행은 학생 1명

문제 7 — 평균점수로 성취도 판정

학생별 평균을 계산한 뒤 집계 결과를 성취도로 분류합니다.

조회 결과

- 학번 · 학생명 · 학과명
- 수강과목수 · 평균점수
- 성취도

작성 조건

- 3개 테이블 LEFT JOIN
- AVG(score)를 CASE로 비교
- 수강 없음은 평가 불가

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 7 솔루션 — 평균점수로 성취도 판정

```
SELECT s.student_no AS 학번, s.name AS 학생명,  
       COALESCE(m.name, '학과 미배정') AS 학과명,  
       COUNT(e.course_id) AS 수강과목수,  
       ROUND(AVG(e.score), 2) AS 평균점수,  
       CASE WHEN AVG(e.score) IS NULL THEN '평가 불가'  
            WHEN AVG(e.score) >= 90 THEN '최우수'  
            WHEN AVG(e.score) >= 85 THEN '우수'  
            WHEN AVG(e.score) >= 80 THEN '보통'  
            ELSE '노력 필요'  
       END AS 성취도  
FROM students s  
LEFT JOIN majors m ON m.id = s.major_id  
LEFT JOIN enrollments e ON e.student_id = s.id  
GROUP BY s.id, s.student_no, s.name, m.name  
ORDER BY 평균점수 DESC NULLS LAST, s.student_no;
```

설명 포인트

- 개별 score가 아닌 AVG(score)
- 학과 NULL은 COALESCE
- 수강 NULL은 평가 불가

문제 8 — 전체 평균보다 높은 학생

학생 평균과 전체 수강 성적 평균을 비교해 조건을 만족한 학생만 출력합니다.

조회 결과

- 학번 · 학생명 · 과목명
- 학생평균 · 전체평균
- 평균과의차이

작성 조건

- 학생별 평균 CTE
- 전체 평균 CTE
- WHERE로 두 평균 비교

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 8 솔루션 — CTE ① 두 평균 계산 , ② 최종 비교

```
WITH student_average AS ( SELECT s.id AS student_id, s.student_no,
                             s.name AS student_name, s.major_id,  AVG(e.score) AS avg_score
                           FROM students s  JOIN enrollments e ON e.student_id = s.id
                           GROUP BY s.id, s.student_no, s.name, s.major_id
                           ),
overall_average AS ( SELECT AVG(score) AS avg_score  FROM enrollments )
SELECT  sa.student_no AS 학번, sa.student_name AS 학생명, COALESCE(m.name, '학과 미배정')
AS 학과명,  ROUND(sa.avg_score, 2) AS 학생평균점수, ROUND(oa.avg_score, 2) AS 전체평균점수,
        ROUND(sa.avg_score - oa.avg_score, 2) AS 평균과의차이
FROM student_average sa
LEFT JOIN majors m ON m.id = sa.major_id
CROSS JOIN overall_average oa
WHERE sa.avg_score > oa.avg_score
ORDER BY 학생평균점수 DESC, sa.student_no;
```

첫 번째 CTE는 학생별 여러 행, 두 번째 CTE는 전체 평균 1행을 만듭니다.
전체 평균은 1행이므로 CROSS JOIN하고, WHERE에서 학생 평균과 비교합니다.

자주 틀리는 다섯 지점을 먼저 점검하세요

JOIN	INNER JOIN으로 미배정 학생 누락	→	LEFT JOIN으로 기준 학생 보존
NULL	= NULL 사용	→	IS NULL 또는 COALESCE
CASE	낮은 점수부터 검사	→	높은 기준부터 순서대로
COUNT	LEFT JOIN 뒤 COUNT(*)	→	NULL이 될 수 있는 연결 열 COUNT
AVG	개별 score로 성취도 판정	→	그룹의 AVG(score) 판정

마지막에는 SQL을 소리 내어 설명해봅니다

- 01 왜 students가 기준 테이블인가?
- 02 왜 LEFT JOIN이 필요한가?
- 03 NULL은 어디에서 생기는가?
- 04 GROUP BY 후 한 행은 무엇을 의미하는가?
- 05 CTE 두 개는 각각 무엇을 계산하는가?

설명할 수 있으면 이해한 것

조회 결과를 재사용하고 다차원 집계를 확장합니다

학사관리 시스템 SQL 5문제해설

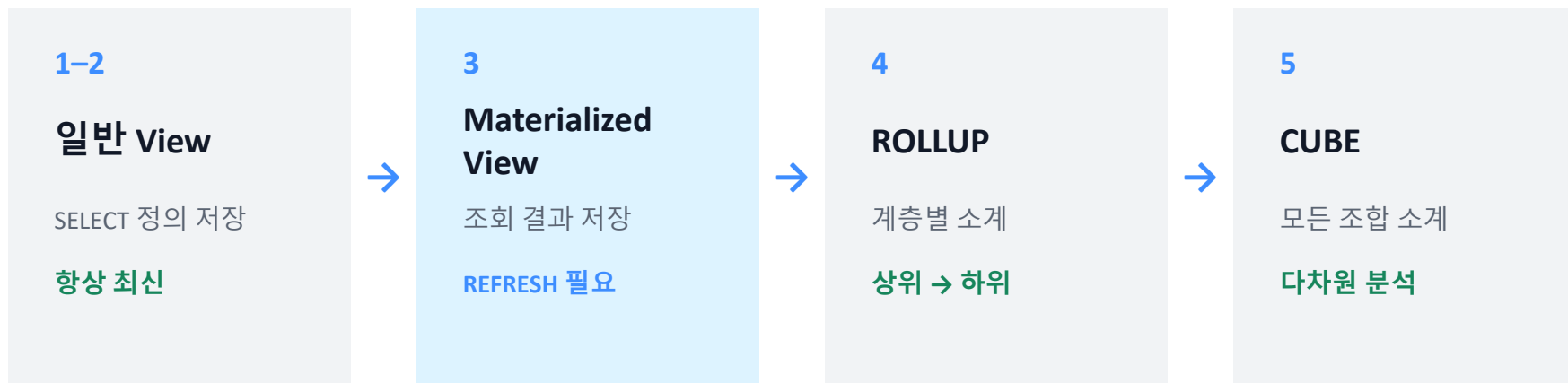
VIEW · MATERIALIZED VIEW · ROLLUP · CUBE

5

ADVANCED
PRACTICE

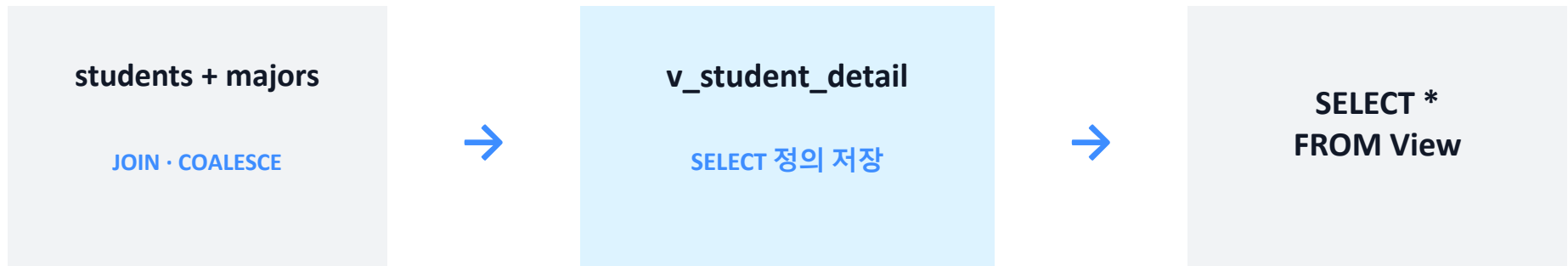
PostgreSQL 13+
app schema

5문제는 “저장 방식”과 “집계 범위”를 확장합니다



핵심 질문: “정의를 저장할까, 결과를 저장할까, 소계를 어디까지 만들까?”

일반 View는 복잡한 SELECT에 이름을 붙입니다



- View 자체에는 데이터가 저장되지 않습니다.
- 원본 테이블이 바뀌면 다음 조회에 바로 반영됩니다.
- 반복되는 JOIN과 열 이름을 단순화합니다.

문제 1 — 학생 상세 정보 View

반복되는 학생·학과 JOIN을 v_student_detail로 재사용합니다.

조회 결과

- 학생 ID · 학번 · 이름 · 이메일
- 학과명 · 학년 · 생성일

작성 조건

- LEFT JOIN으로 전체 학생
- 학과 NULL은 COALESCE
- View 생성 후 학번 정렬

제공파일 :

day02_postgresql_app_students_enrollments_샘플_스키마.sql

day02_postgresql_app_students_enrollments_DQL_문제02_5문항_학습용.sql

문제 1 솔루션 — 학생 상세 View 생성·조회

```
DROP VIEW IF EXISTS v_student_detail CASCADE;

CREATE VIEW v_student_detail AS
SELECT s.id AS student_id, s.student_no, s.name AS student_name, s.email,
       COALESCE(m.name, '학과 미배정') AS major_name, s.grade, s.created_at
FROM students s
LEFT JOIN majors m ON m.id = s.major_id;

SELECT *
FROM v_student_detail
ORDER BY student_no;
```

View에는 데이터가 아니라 SELECT 정의가 저장되며 원본 변경이 다음 조회에 반영됩니다.

문제 2 — 과목별 성적 통계 View

모든 과목의 수강 인원과 평균·최고·최저 점수를 한 행으로 집계합니다.

조회 결과

- 과목 ID · 코드 · 이름 · 학점
- 수강생수 · 평균 · 최고 · 최저

작성 조건

- 수강생 없는 과목 포함
- 과목 열로 GROUP BY
- 평균 NULL은 마지막

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 2 솔루션 — 과목별 통계 View 생성·조회

```
DROP VIEW IF EXISTS v_course_score_stats CASCADE;

CREATE VIEW v_course_score_stats AS
SELECT c.id AS course_id, c.course_code, c.name AS course_name, c.credit,
       COUNT(e.student_id) AS student_count, ROUND(AVG(e.score), 2) AS avg_score,
       MAX(e.score) AS max_score, MIN(e.score) AS min_score
FROM courses c LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.course_code, c.name, c.credit;

SELECT * FROM v_course_score_stats
ORDER BY avg_score DESC NULLS LAST, course_id;
```

COUNT(e.student_id)는 LEFT JOIN으로 남은 NULL 행을 제외합니다.
NULLS LAST는 수강생이 없어 평균점수가 NULL인 과목을 결과 마지막에 배치합니다.

문제 3 — 학과별 요약 Materialized View

학과별 학생 수·수강 건수·평균점수를 실제 결과로 저장합니다.

조회 결과

- major_key · 학과명
- 학생수 · 수강건수 · 평균

작성 조건

- 미배정 학과 key는 0
- 학생수는 DISTINCT
- UNIQUE INDEX와 REFRESH

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 3 솔루션 — Materialized View 생성 ①

```
DROP MATERIALIZED VIEW IF EXISTS mv_major_learning_summary CASCADE;

CREATE MATERIALIZED VIEW mv_major_learning_summary AS
SELECT COALESCE(m.id, 0) AS major_key,
       COALESCE(m.name, '학과 미배정') AS major_name,
       COUNT(DISTINCT s.id) AS student_count,
       COUNT(e.course_id) AS enrollment_count,
       ROUND(AVG(e.score), 2) AS avg_score
FROM students s
LEFT JOIN majors m ON m.id = s.major_id
LEFT JOIN enrollments e ON e.student_id = s.id
```

JOIN 후 학생이 수강 건수만큼 반복되므로 학생 수에는 COUNT(DISTINCT s.id)를 사용합니다.

문제 3 솔루션 — Materialized View 생성 ②

```
GROUP BY
  COALESCE(m.id, 0),
  COALESCE(m.name, '학과 미배정')
WITH DATA;

CREATE UNIQUE INDEX ux_mv_major_learning_summary_major_key
  ON mv_major_learning_summary(major_key);

SELECT *
FROM mv_major_learning_summary
ORDER BY student_count DESC, major_key;
```

WITH DATA는 생성과 동시에 조회 결과를 저장합니다.

문제 3 솔루션 — Materialized View 새로고침

-- 일반 새로고침

```
REFRESH MATERIALIZED VIEW mv_major_learning_summary;
```

-- 조회 차단을 줄이는 동시 새로고침

```
REFRESH MATERIALIZED VIEW CONCURRENTLY  
mv_major_learning_summary;
```

CONCURRENTLY 방식에는 행을 유일하게 식별하는 적합한 UNIQUE INDEX가 필요합니다.

문제 4 — 학과 → 학년별 ROLLUP

학과·학년 상세, 학과 소계, 전체 합계를 계층적으로 출력합니다.

조회 결과

- 학과명 · 학년
- 학생수

작성 조건

- ROLLUP(학과, 학년)
- GROUPING으로 합계 구분
- 미배정 학과 유지

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 4 솔루션 — ROLLUP 전체 쿼리

```
SELECT
  CASE      WHEN GROUPING(m.name) = 1 THEN '전체 합계'
            ELSE COALESCE(m.name, '학과 미배정')
  END AS major_name,
  CASE      WHEN GROUPING(s.grade) = 1 THEN '학과 소계'
            ELSE s.grade::text || '학년'
  END AS grade, COUNT(s.id) AS student_count
FROM students s LEFT JOIN majors m ON m.id = s.major_id
GROUP BY ROLLUP(m.name, s.grade)
ORDER BY GROUPING(m.name), m.name NULLS LAST,
         GROUPING(s.grade), s.grade NULLS LAST;
```

ROLLUP(A,B)는 (A,B) → (A) → ()를 만들며 B만의 소계는 만들지 않습니다.

문제 5 — 학과 × 과목 CUBE

학과와 과목의 모든 조합별 수강 건수와 평균점수를 출력합니다.

조회 결과

- 학과명 · 과목명
- 수강건수 · 평균점수

작성 조건

- 상세·학과소계·과목소계·전체
- GROUPING으로 합계 표시
- CUBE(학과, 과목)

정답을 보기 전에 FROM의 기준 테이블과 JOIN 종류를 먼저 생각해 보세요.

문제 5 솔루션 — CUBE 전체 쿼리 ①

```
SELECT
  CASE    WHEN GROUPING(m.name) = 1 THEN '모든 학과'
          ELSE COALESCE(m.name, '학과 미배정')
  END AS major_name,
  CASE    WHEN GROUPING(c.name) = 1 THEN '모든 과목'
          ELSE c.name
  END AS course_name,
  COUNT(*) AS enrollment_count,  ROUND(AVG(e.score), 2) AS avg_score
FROM enrollments e JOIN students s ON s.id = e.student_id
LEFT JOIN majors m ON m.id = s.major_id
JOIN courses c ON c.id = e.course_id
```

기준이 enrollments이므로 한 행은 실제 수강 1건이며 COUNT(*)를 사용할 수 있습니다.

문제 5 솔루션 — CUBE 전체 쿼리 ②

```
GROUP BY CUBE(m.name, c.name)
ORDER BY
  GROUPING(m.name),
  m.name NULLS LAST,
  GROUPING(c.name),
  c.name NULLS LAST;
```

CUBE(A,B)는 (A,B), (A), (B), () 네 집계를 모두 생성합니다.

ROLLUP과 CUBE는 필요한 소계 범위가 다릅니다

구분	ROLLUP(A, B)	CUBE(A, B)
상세 (A,B)	✓	✓
A 소계	✓	✓
B 소계	—	✓
전체 합계	✓	✓
적합한 분석	계층형	다차원

설명할 수 있어야 할 여섯 가지를 확인합니다

- 01 View에는 데이터가 저장되는가?
- 02 Materialized View는 언제 최신화되는가?
- 03 학생 수에 DISTINCT가 필요한 이유는?
- 04 WITH DATA는 무엇을 의미하는가?
- 05 GROUPING()은 왜 필요한가?
- 06 ROLLUP과 CUBE의 소계 차이는?

말로 설명하면 완성

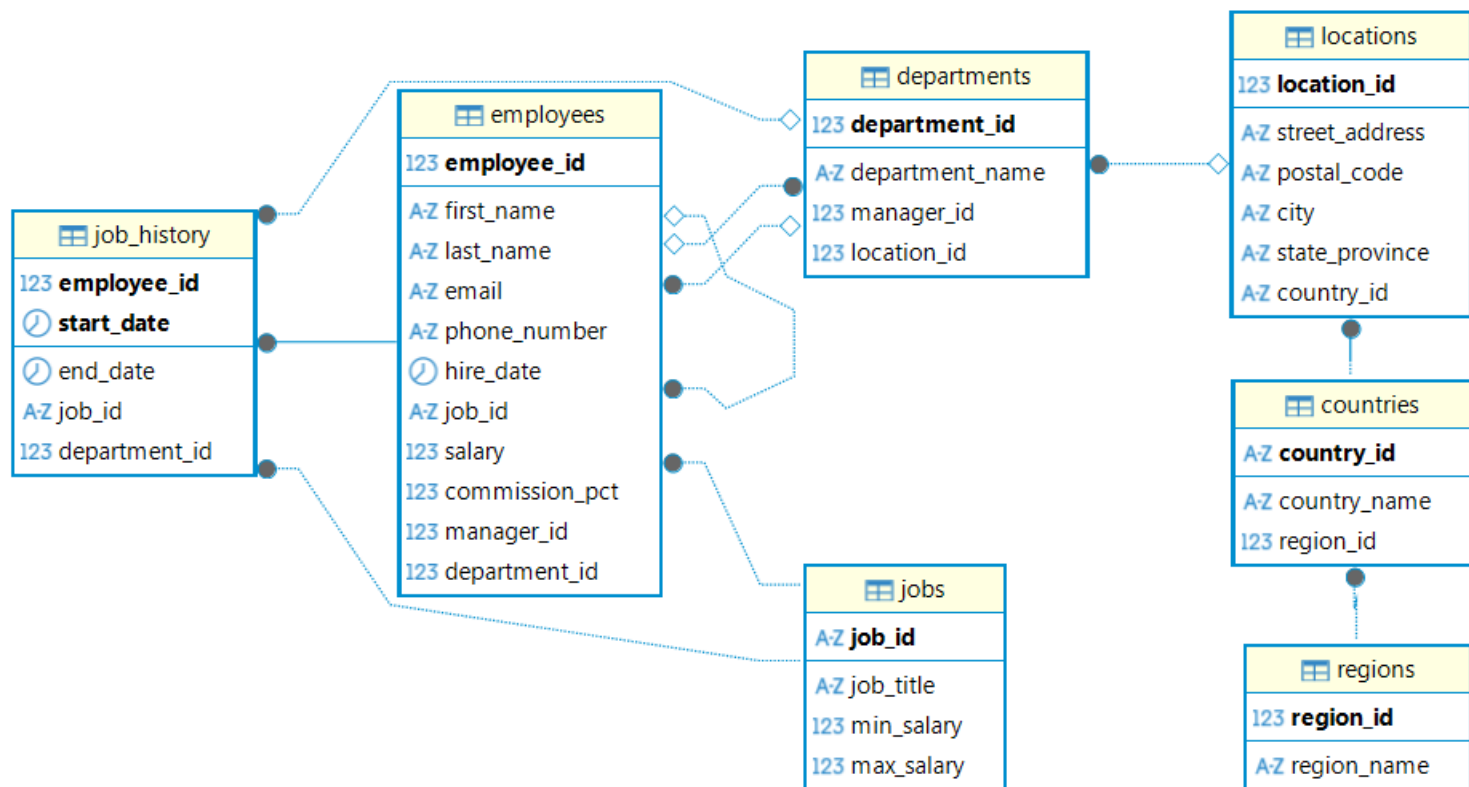
PostgreSQL HR 샘플 데이터

제공파일 : day02_PostgreSQL_HR_emp_dept_샘플_스키마.sql

DDL
DML

PostgreSQL 13+
hr schema

종합실습 HR SCHEMA - ERD



HR 샘플 데이터로 해결하는 PostgreSQL 종합실습

39문제 · 문제 전용 평가자료

39

PROBLEMS
ONLY

PostgreSQL 13+
hr schema

종합실습 및 제출 안내

- 문제(day02_PostgreSQL_HR_종합실습_문제.sql)를 보고 SQL 답안을 작성합니다.
- DBeaver에서 항목별로 실행하여 결과를 확인합니다.
- 실행결과를 캡처합니다. 제출양식 : day02_캡처_작성자명.pdf 파일에 모아서 제출합니다.
- 주어진 sql문제파일에 정답 쿼리를 작성해서로 제출합니다. 제출양식 : day02_쿼리_작성자명.sql
- 제출 경로: Slack 다이렉트 메시지

주의: VIEW와 MATERIALIZED VIEW 문제는 객체를 생성하므로 순서대로 실행하십시오.

실습 환경과 공통 규칙

대상 database: skala_db
대상 SCHEMA: hr

검색 경로

```
SET search_path TO hr, public;
```

별칭 사용

영문·한글 모두 가능

단, 결과를 쉽게 이해할 수 있도록
가독성 있는 이름을 부여합니다.

정답 SQL은 이 자료에 포함되어 있지 않습니다.

39문제는 여덟 분야로 구성됩니다

10

GROUP BY

그룹 집계와 HAVING을 활용합니다.

5

서브쿼리

다른 쿼리의 결과를 조건으로 사용합니다.

3

CTE

계산 단계를 WITH 절로 분리합니다.

10

JOIN

관계와 보존할 행에 맞는 JOIN을 선택합니다.

5

VIEW

자주 쓰는 SELECT 정의를 View로 생성합니다.

2

MATERIALIZED VIEW

조회 결과를 저장하고 새로고침합니다.

2

CUBE

모든 차원 조합별 소계와 합계를 만듭니다.

2

ROLLUP

계층 순서에 따른 소계와 합계를 만듭니다.

총 39문제 · 솔루션 없이 문제와 조건만 제공합니다.

10 PROBLEMS

A. GROUP BY

그룹 집계와 HAVING을 활용합니다.

COUNT · SUM · AVG

CASE 그룹

HAVING

A. GROUP BY 1 — 부서별 사원 수

문제

HR 데이터를 활용하여 '부서별 사원 수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. department_id가 같은 사원을 그룹화하여 인원수를 계산합니다. 부서 미배정 사원은 '부서 미배정'으로 표시합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 2 — 직무별 사원 수와 평균 급여

문제

HR 데이터를 활용하여 '직무별 사원 수와 평균 급여' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. 직무별 인원수와 평균 급여를 동시에 집계합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 3 — 부서별 급여 합계·평균·최솟값·최댓값

문제

HR 데이터를 활용하여 '부서별 급여 합계·평균·최솟값·최댓값' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. 하나의 그룹에 여러 집계 함수를 적용합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 4 — 관리자별 부하 직원 수

문제

HR 데이터를 활용하여 '관리자별 부하 직원 수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. 사원 테이블을 관리자 기준으로 그룹화합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 5 — 입사 연도별 입사자 수

문제

HR 데이터를 활용하여 '입사 연도별 입사자 수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. EXTRACT로 날짜에서 연도만 꺼낸 뒤 그룹화합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 6 — 커미션 수령 여부별 인원과 평균 급여

문제

HR 데이터를 활용하여 '커미션 수령 여부별 인원과 평균 급여' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. CASE 표현식의 결과도 GROUP BY 기준으로 사용할 수 있습니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 7 — 국가별 근무 인원 수

문제

HR 데이터를 활용하여 '국가별 근무 인원 수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. employees부터 countries까지 여러 테이블을 연결한 뒤 국가별로 집계합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 8 — 부서별 직무 종류 수

문제

HR 데이터를 활용하여 '부서별 직무 종류 수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. COUNT(DISTINCT 열)을 사용하면 중복을 제외한 개수를 구할 수 있습니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 9 — 평균 급여가 8,000 이상인 부서

문제

HR 데이터를 활용하여 '평균 급여가 8,000 이상인 부서' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. WHERE는 그룹화 전 행을, HAVING은 그룹화 후 결과를 필터링합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

A. GROUP BY 10 — 부서별 급여 구간 인원수

문제

HR 데이터를 활용하여 '부서별 급여 구간 인원수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY와 필요한 집계 함수를 사용하십시오. CASE로 만든 급여 구간과 부서를 두 가지 기준으로 그룹화합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

B. 서브쿼리

다른 쿼리의 결과를 조건으로 사용합니다.

단일 행

상관 서브쿼리

EXISTS

B. 서브쿼리 1 — 전체 평균보다 급여가 높은 사원

문제

HR 데이터를 활용하여 '전체 평균보다 급여가 높은 사원' 결과를 조회하는 SQL을 작성하십시오.

조건

서브쿼리를 사용해 조건을 해결하십시오. 단일 행 서브쿼리에서 계산한 전체 평균과 각 사원의 급여를 비교합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

B. 서브쿼리 2 — 소속 부서 평균보다 급여가 높은 사원

문제

HR 데이터를 활용하여 '소속 부서 평균보다 급여가 높은 사원' 결과를 조회하는 SQL을 작성하십시오.

조건

서브쿼리를 사용해 조건을 해결하십시오. 바깥 쿼리의 department_id를 참조하는 상관 서브쿼리입니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

B. 서브쿼리 3 — 사원이 한 명이라도 존재하는 부서

문제

HR 데이터를 활용하여 '사원이 한 명이라도 존재하는 부서' 결과를 조회하는 SQL을 작성하십시오.

조건

서브쿼리를 사용해 조건을 해결하십시오. EXISTS는 서브쿼리 결과 행의 존재 여부만 확인합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

B. 서브쿼리 4 — 가장 높은 평균 급여를 가진 부서

문제

HR 데이터를 활용하여 '가장 높은 평균 급여를 가진 부서' 결과를 조회하는 SQL을 작성하십시오.

조건

서브쿼리를 사용해 조건을 해결하십시오. FROM 절 서브쿼리로 부서 평균을 만든 후 가장 큰 평균과 비교합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

B. 서브쿼리 5 — 과거 직무 이력이 없는 사원

문제

HR 데이터를 활용하여 '과거 직무 이력이 없는 사원' 결과를 조회하는 SQL을 작성하십시오.

조건

서브쿼리를 사용해 조건을 해결하십시오. NOT EXISTS는 관련 행이 하나도 없는 대상을 찾을 때 안전합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

C. CTE

계산 단계를 WITH 절로 분리합니다.

WITH

다단계 CTE

재귀 CTE

C. CTE 1 — 부서별 평균 급여와 전체 평균 비교

문제

HR 데이터를 활용하여 '부서별 평균 급여와 전체 평균 비교' 결과를 조회하는 SQL을 작성하십시오.

조건

WITH 절(CTE)을 사용해 단계적으로 작성하십시오. WITH 절에서 부서 통계를 먼저 만든 뒤 메인 쿼리에서 사용합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

C. CTE 2 — 두 단계 CTE로 부서별 최고 급여자 찾기

문제

HR 데이터를 활용하여 '두 단계 CTE로 부서별 최고 급여자 찾기' 결과를 조회하는 SQL을 작성하십시오.

조건

WITH 절(CTE)을 사용해 단계적으로 작성하십시오. 첫 번째 CTE의 결과를 두 번째 CTE가 다시 참조합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

C. CTE 3 — 재귀 CTE로 사원 조직도 조회

문제

HR 데이터를 활용하여 '재귀 CTE로 사원 조직도 조회' 결과를 조회하는 SQL을 작성하십시오.

조건

WITH 절(CTE)을 사용해 단계적으로 작성하십시오. 최고경영자를 시작점으로 manager_id 관계를 반복하여 내려 갑니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

10 PROBLEMS

D. JOIN

관계와 보존할 행에 맞는 JOIN을 선택합니다.

INNER

OUTER

SELF · 비등가

D. JOIN 1: INNER JOIN — 사원과 부서

문제

HR 데이터를 활용하여 '사원과 부서' 결과를 조회하는 SQL을 작성하십시오.

조건

INNER JOIN 방식을 활용하십시오. 부서가 배정된 사원만 출력합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 2: INNER JOIN — 사원과 직무

문제

HR 데이터를 활용하여 '사원과 직무' 결과를 조회하는 SQL을 작성하십시오.

조건

INNER JOIN 방식을 활용하십시오. job_id가 일치하는 직무명을 사원 정보에 결합합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 3: 4개 테이블 INNER JOIN — 사원의 부서·도시·국가

문제

HR 데이터를 활용하여 '사원의 부서·도시·국가' 결과를 조회하는 SQL을 작성하십시오.

조건

4개 테이블 INNER JOIN 방식을 활용하십시오. 사원에서 국가까지 FK 관계를 차례로 연결합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 4: SELF JOIN — 사원과 직속 관리자

문제

HR 데이터를 활용하여 '사원과 직속 관리자' 결과를 조회하는 SQL을 작성하십시오.

조건

SELF JOIN 방식을 활용하십시오. 동일한 employees 테이블을 사원용 e와 관리자용 m으로 두 번 사용합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 5: LEFT OUTER JOIN — 부서 미배정 사원 포함

문제

HR 데이터를 활용하여 '부서 미배정 사원 포함' 결과를 조회하는 SQL을 작성하십시오.

조건

LEFT OUTER JOIN 방식을 활용하십시오. 왼쪽 employees의 모든 행을 보존하므로 부서가 없는 사원도 나옵니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 6: LEFT OUTER JOIN — 사원이 없는 부서 포함

문제

HR 데이터를 활용하여 '사원이 없는 부서 포함' 결과를 조회하는 SQL을 작성하십시오.

조건

LEFT OUTER JOIN 방식을 활용하십시오. 모든 부서를 보존하고 소속 사원이 없으면 사원 열을 NULL로 표시합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 7: RIGHT OUTER JOIN — 모든 직무 포함

문제

HR 데이터를 활용하여 '모든 직무 포함' 결과를 조회하는 SQL을 작성하십시오.

조건

RIGHT OUTER JOIN 방식을 활용하십시오. 오른쪽 jobs의 모든 행을 보존하여 현재 담당 사원이 없는 직무도 보여줍니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 8: FULL OUTER JOIN — 모든 부서와 모든 사원

문제

HR 데이터를 활용하여 '모든 부서와 모든 사원' 결과를 조회하는 SQL을 작성하십시오.

조건

FULL OUTER JOIN 방식을 활용하십시오. 어느 쪽에도 일치 상대가 없는 행까지 모두 보존합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. D. JOIN 9: INNER JOIN — 사원의 과거 직무 이력

문제

HR 데이터를 활용하여 '사원의 과거 직무 이력' 결과를 조회하는 SQL을 작성하십시오.

조건

INNER JOIN 방식을 활용하십시오. job_history를 employees, jobs, departments와 결합합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

D. JOIN 10: 비등가 JOIN — 사원의 급여가 직무 급여 범위에 맞는지 확인

문제

HR 데이터를 활용하여 '사원의 급여가 직무 급여 범위에 맞는지 확인' 결과를 조회하는 SQL을 작성하십시오.

조건

비등가 JOIN 방식을 활용하십시오. 등호가 아닌 BETWEEN 조건으로 급여와 직무 범위를 결합합니다.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

E. VIEW

자주 쓰는 SELECT 정의를 View로 생성합니다.

CREATE VIEW

DROP VIEW

조회 확인

E. VIEW 1 — 사원 기본 정보 View

문제

HR 데이터를 활용하여 '사원 기본 정보 View'를 제공하는 일반 View를 생성하고 조회하십시오.

조건

지정된 이름의 일반 View를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

E. VIEW 2 — 사원·부서·직무 통합 View

문제

HR 데이터를 활용하여 '사원·부서·직무 통합 View'를 제공하는 일반 View를 생성하고 조회하십시오.

조건

지정된 이름의 일반 View를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

E. VIEW 3 — 부서별 급여 통계 View

문제

HR 데이터를 활용하여 '부서별 급여 통계 View'를 제공하는 일반 View를 생성하고 조회하십시오.

조건

지정된 이름의 일반 View를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

E. VIEW 4 — 관리자별 부하 직원 View

문제

HR 데이터를 활용하여 '관리자별 부하 직원 View'를 제공하는 일반 View를 생성하고 조회하십시오.

조건

지정된 이름의 일반 View를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

E. VIEW 5 — 직무 이력 상세 View

문제

HR 데이터를 활용하여 '직무 이력 상세 View'를 제공하는 일반 View를 생성하고 조회하십시오.

조건

지정된 이름의 일반 View를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

F. MATERIALIZED VIEW

조회 결과를 저장하고 새로고침합니다.

WITH DATA

UNIQUE INDEX

REFRESH

F. Materialized View 1 — 부서별 급여 요약

문제

HR 데이터를 활용하여 '부서별 급여 요약' Materialized View를 생성하고 조회하십시오.

조건

Materialized View와 UNIQUE INDEX를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

F. Materialized View 2 — 국가별 사원 현황

문제

HR 데이터를 활용하여 '국가별 사원 현황' Materialized View를 생성하고 조회하십시오.

조건

Materialized View와 UNIQUE INDEX를 생성한 뒤 결과를 조회하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

G. CUBE

모든 차원 조합별 소계와 합계를 만듭니다.

CUBE

GROUPING

네 집계 조합

G. CUBE 1 — 부서와 직무별 급여 합계

문제

HR 데이터를 활용하여 '부서와 직무별 급여 합계' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY CUBE와 GROUPING 함수를 사용해 상세·소계·전체 합계를 구하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

G. CUBE 2 — 입사 연도와 커미션 수령 여부별 인원수

문제

HR 데이터를 활용하여 '입사 연도와 커미션 수령 여부별 인원수' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY CUBE와 GROUPING 함수를 사용해 상세·소계·전체 합계를 구하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

H. ROLLUP

계층 순서에 따른 소계와 합계를 만듭니다.

ROLLUP

GROUPING

계층 소계

H. ROLLUP 1 — 국가 → 도시 → 부서 계층별 급여 합계

문제

HR 데이터를 활용하여 '국가 → 도시 → 부서 계층별 급여 합계' 결과를 조회하는 SQL을 작성하십시오.

조건

GROUP BY ROLLUP과 GROUPING 함수를 사용해 계층별 상세·소계·전체 합계를 구하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

H. ROLLUP 2 — 입사 연도 → 부서별 인원 및 급여 합계

문제

HR 데이터를 활용하여 '입사 연도 → 부서별 인원 및 급여 합계' 결과를 조회하는 SQL을 작성하십시오.

.

조건

GROUP BY ROLLUP과 GROUPING 함수를 사용해 계층별 상세·소계·전체 합계를 구하십시오.

TODO

요구사항을 만족하는 SQL을 작성하고 실행 결과를 캡처하십시오.

제출 전 최종 확인

- 39문제(A~H)의 SQL을 모두 작성했는가?
- 각 쿼리가 오류 없이 실행되는가?
- 요구된 NULL·소계·정렬 조건이 반영되었는가?
- 문제별 실행 결과를 캡처했는가?

제출방법 :

Slack – 다이렉트메세지

- day02_캡처_작성자명.pdf
- day02_쿼리_작성자명.sql



수고하셨습니다.

본 문서는 SK㈜ AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.